

```
In [1]: import numpy as np
import pandas as pd
import scipy

import statsmodels
import matplotlib as mpl
import matplotlib.pyplot as plt
import seaborn as sns

import warnings
warnings.filterwarnings(action = 'ignore')

from IPython.display import Image

for i in [pd, np, scipy, statsmodels, mpl, sns]:
    print(i.__name__, i.__version__)
```

```
pandas 2.1.4
numpy 1.26.4
scipy 1.11.4
statsmodels 0.14.0
matplotlib 3.8.0
seaborn 0.12.2
```

2-7 회귀 분석

```
In [2]: Image('98.png')
```

Out[2]:

0. 데이터셋 소개

Wild blueberry

[Wild blueberry](#) 야생 블루베리의 수확량을 시뮬레이션한 데이터셋입니다.

Features	Unit	Description
Row#		Index
clonesize	m2	The average blueberry clone size in the field
honeybee	bees/m2/min	Honeybee density in the field
bumbles	bees/m2/min	Bumblebee density in the field
andrena	bees/m2/min	Andrena bee density in the field
osmia	bees/m2/min	Osmia bee density in the field
MaxOfUpperTRange	°C	The highest record of the upper band daily air temperature during the bloom season
MinOfUpperTRange	°C	The lowest record of the upper band daily air temperature
AverageOfUpperTRange	°C	The average of the upper band daily air temperature
MaxOfLowerTRange	°C	The highest record of the lower band daily air temperature
MinOfLowerTRange	°C	The lowest record of the lower band daily air temperature
AverageOfLowerTRange	°C	The average of the lower band daily air temperature
RainingDays	Day	The total number of days during the bloom season, each of which has precipitation larger than zero
AverageRainingDays	Day	The average of raining days of the entire bloom season
fruitset		The amount of fruit per tree
fruitmass		The mass of a fruit
seeds		The mass of a seed
yield		The amount of yield

```
In [3]: df_berry = pd.read_csv('data/wild_blueberry.csv')
df_berry.head()
```

```
Out[3]:
```

	Row#	clonesize	honeybee	bumbles	andrena	osmia	MaxOfUpperTRange	MinOfUpperTRange	AverageOfUpperTRange	MaxOfLowe
0	0	37.5	0.75	0.25	0.25	0.25	86.0	52.0	71.9	
1	1	37.5	0.75	0.25	0.25	0.25	86.0	52.0	71.9	
2	2	37.5	0.75	0.25	0.25	0.25	94.6	57.2	79.0	
3	3	37.5	0.75	0.25	0.25	0.25	94.6	57.2	79.0	
4	4	37.5	0.75	0.25	0.25	0.25	86.0	52.0	71.9	



```
In [3]: Image('99.png')
```

Out[3]: **1. 선형 회귀(Linear Regression)**

구성 요소

- 표현법 1: 수열 형태

$$\{y_i, x_{i1}, \dots, x_{ip}\}_{i=1}^n$$

$$y_i = \beta_0 + \beta_1 x_{i1} + \beta_2 x_{i2} + \dots + \beta_p x_{ip} + \epsilon_i = x_i^T \beta + \epsilon_i$$

- 표현법 2: 벡터 형태

$$\mathbf{y} = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{bmatrix}, \mathbf{\epsilon} = \begin{bmatrix} \epsilon_1 \\ \epsilon_1 \\ \vdots \\ \epsilon_n \end{bmatrix}, \mathbf{x}_i = \begin{bmatrix} 1 \\ x_{i1} \\ \vdots \\ x_{ip} \end{bmatrix}, \boldsymbol{\beta} = \begin{bmatrix} \beta_0 \\ \beta_1 \\ \vdots \\ \beta_p \end{bmatrix}$$

$$\mathbf{X} = \begin{bmatrix} x_0^T \\ x_1^T \\ \vdots \\ x_n^T \end{bmatrix} = \begin{bmatrix} 1 & x_{11} & x_{12} & \dots & x_{1p} \\ 1 & x_{21} & x_{22} & \dots & x_{2p} \\ \vdots & \vdots & \ddots & \ddots & \vdots \\ 1 & x_{n1} & x_{n2} & \dots & x_{np} \end{bmatrix}$$

$$\mathbf{y} = \mathbf{X}\boldsymbol{\beta} + \boldsymbol{\epsilon}$$

y : 종속 변수(dependent variable), 내생 변수(endogenous variable, 모델 내에서 다른 변수의 영향을 받는 변수), 대상 변수(target variable), 반응 변수

X : 독립 변수(independent variable), 외생 변수(exogenous variable, 모델 내에서 설명되지 않는 변수), 예측 변수(predictor variable), 입력 변수(input variable)

β : 회귀 계수(regression coefficients)

ϵ : 오차

선형 회귀 모델의 주요 가정

선형성 (Linearity): 종속 변수와 독립 변수 간의 관계가 선형이어야 합니다.

독립성(independence): 각각의 관측은 서로 독립입니다.

어떤 관측치의 오차가 다른 관측치의 오차에 영향을 미치지 않아야 합니다.

등분산(Homoscedasticity): 오차의 분산은 독립 변수와 상관없이 일정해야 합니다.

정규성 (Normality of Residuals): 오차는 정규분포를 따라야 합니다.

완전한 다중공선성 부재 (Lack of perfect multicollinearity):

독립 변수 각각을 종속변수로 했을 때 예측 오차가 없이

완전히 설명이 가능한 변수가 존재하지 않아야 합니다.

In [4]: `Image('100.png')`

Out[4]: 회귀 계수의 추정

최소 제곱법 (Least Square Method, Ordinary Least Square)

$$S(\beta) = \sum_{i=1}^n (y_i - x_i^T \beta)^2 = \|y - X\beta\|^2$$

$$\hat{\beta} = \operatorname{argmin}_{\beta} S(\beta)$$

$$\|y - X\beta\|^2 = (y - X\beta)^T (y - X\beta)$$

β 의 추정치를 $\hat{\beta}$ 이라고 합니다.

$$\frac{dS(\hat{\beta})}{d\hat{\beta}} = 0$$

$$\frac{dS(\hat{\beta})}{d\hat{\beta}} = \frac{d(y^T y - y^T X \hat{\beta} - \hat{\beta}^T X^T y + \hat{\beta}^T X^T X \hat{\beta})}{d\hat{\beta}}$$

$$\hat{\beta}^T X^T y \text{는 } 1 \times 1 \text{ 행렬이므로 } \hat{\beta}^T X^T y = (\hat{\beta}^T X^T y)^T$$

$$\hat{\beta}^T X^T y = y^T X \hat{\beta}$$

$$\frac{dS(\hat{\beta})}{d\hat{\beta}} = \frac{d(y^T y - 2y^T X \hat{\beta} + \hat{\beta}^T X^T X \hat{\beta})}{d\hat{\beta}}$$

$$\frac{dy^T X \hat{\beta}}{d\hat{\beta}} = (y^T X)^T = X^T y$$

$$\frac{d\hat{\beta}^T X^T X \hat{\beta}}{d\hat{\beta}} = 2X^T X \hat{\beta}$$

$$\frac{dS(\hat{\beta})}{d\hat{\beta}} = -2X^T y + 2X^T X \hat{\beta}$$

$$-2X^T y + 2X^T X \hat{\beta} = 0$$

$$\hat{\beta} = (X^T X)^{-1} X^T y$$

회귀 계수의 의미

회귀 계수($\beta_i, i > 0$): 독립 변수 X_i 가 1 증가 할 때, 종속 변수의 변화량

상수항(intercept, β_0): 모든 독립 변수가 0일 때 종속 변수의 평균 $\leftarrow$ 모든 독립 변수가 0인 상황에 의미를 부여 할 수 있다면 분석의 대상이 되겠지만, 보통 독립 변수가 0 인 상황이 특별한 의미를 가지지 못한 경우가 많으므로 분석에 대상이 되지는 못합니다.

[Ex.1]

단순 선형 회귀(Simple Linear Regression) - 독립 변수의 수가 하나인 선형 회귀 모델

df_berry에서

clonesize == 12.5 and honeybee == 0.25 and

bumbles == 0.25 and AverageOfUpperTRange == 71.9 and

AverageOfLowerTRange == 50.8

인 관측 데이터만 선택하여 df_berry2를 만듭니다.

df_berry2에서 yield 는 종속 변수 fruitset은 독립 변수로 회귀 모델을 만들어 봅니다.

```
In [4]: df_berry2 = df_berry.query(
        'clonesize == 12.5 and honeybee == 0.25 and bumbles == 0.25 and AverageOfUpperTRange == 71.9 and AverageOfLowerTRange == 50.8')
df_berry2
```

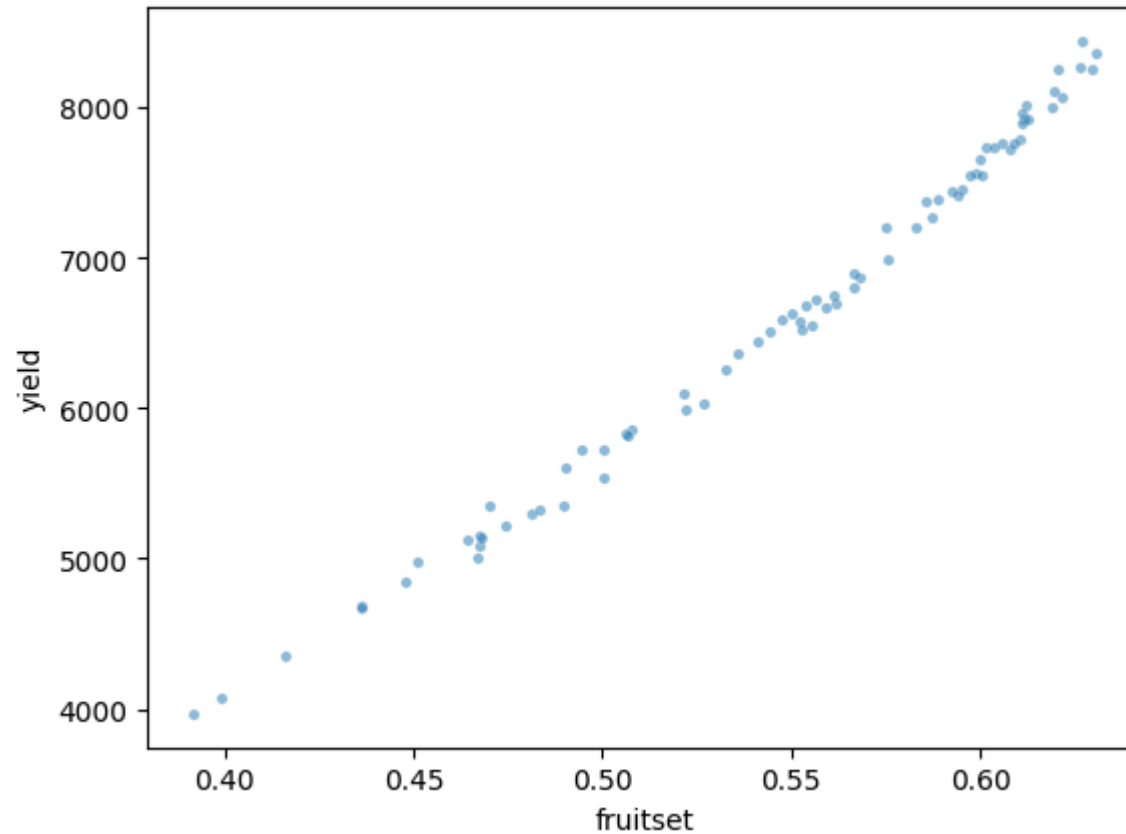
Out[4]:

	Row#	clonesize	honeybee	bumbles	andrena	osmia	MaxOfUpperTRange	MinOfUpperTRange	AverageOfUpperTRange	MaxOfLo
51	51	12.5	0.25	0.25	0.25	0.25	86.0	52.0	71.9	
52	52	12.5	0.25	0.25	0.25	0.25	86.0	52.0	71.9	
55	55	12.5	0.25	0.25	0.25	0.25	86.0	52.0	71.9	
56	56	12.5	0.25	0.25	0.25	0.25	86.0	52.0	71.9	
67	67	12.5	0.25	0.25	0.25	0.38	86.0	52.0	71.9	
...	...	...	...	...	...	...	...	...	...	...
328	328	12.5	0.25	0.25	0.75	0.63	86.0	52.0	71.9	
339	339	12.5	0.25	0.25	0.75	0.75	86.0	52.0	71.9	
340	340	12.5	0.25	0.25	0.75	0.75	86.0	52.0	71.9	
343	343	12.5	0.25	0.25	0.75	0.75	86.0	52.0	71.9	
344	344	12.5	0.25	0.25	0.75	0.75	86.0	52.0	71.9	

76 rows × 18 columns



```
In [5]: sns.scatterplot(df_berry2, x='fruitset', y='yield', s = 15, alpha=0.5)
plt.show()
```



```
In [6]: # 절편을 따로 계산하지 않고 포함하여 구하기 위해 모든 값이 1인 열을 추가합니다.
X = df_berry2[['fruitset']].assign(
    const = 1
)
y = df_berry2['yield']
# 정규 방정식을 이용하여 회귀계수를 계산합니다.
# np.linalg.inv : 역행렬을 구합니다.
# np.ndarray.dot: 벡터일때는 내적, 행렬일 때는 행렬곱을 계산합니다.
beta_hat = np.linalg.inv(X.T.dot(X)).dot(X.T).dot(y)
beta_hat
```

```
Out[6]: array([18664.94491423, -3595.388558  ])
```

```
In [7]: df_berry[['fruitset']].agg(['min', 'max']).assign(const=1).dot(beta_hat)
```

```
Out[7]: min      1.937222
        max    8576.844939
        dtype: float64
```

```
In [8]: '''
코드는 seaborn 라이브러리를 사용하여 산점도를 그리고,
선형 회귀 모델의 예측 값을 오렌지색 선으로 그리는 작업을 수행합니다.

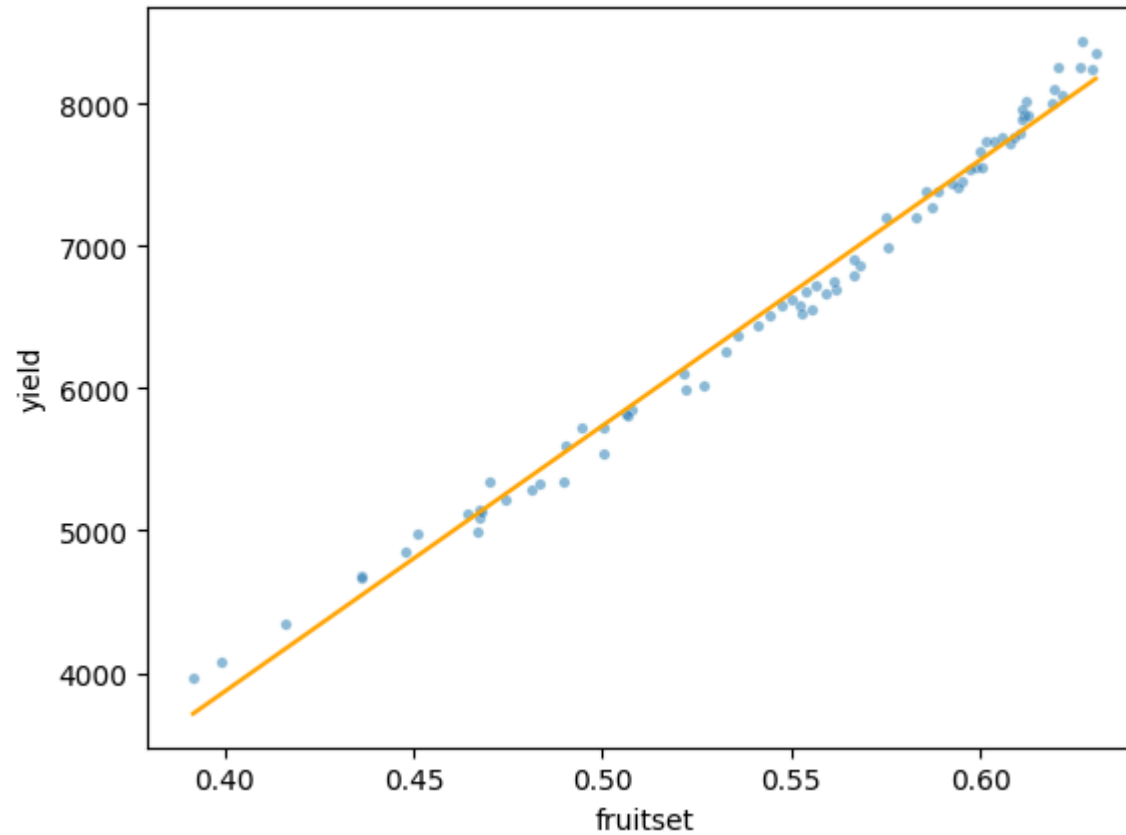
1. `sns.scatterplot(df_berry2, x='fruitset', y='yield', s=15, alpha=0.5)` :
   seaborn 라이브러리의 `scatterplot` 함수를 사용하여
   산점도를 그립니다.
   `df_berry2`는 데이터프레임을 나타내며, 'fruitset' 열을 x축으로,
   'yield' 열을 y축으로 설정하여 산점도를 생성합니다.
   `s=15`는 점의 크기를 설정하고, `alpha=0.5`는 점의 투명도를 설정합니다.

2. `plt.plot(df_berry2[['fruitset']].agg(['min', 'max']),
   df_berry2[['fruitset']].agg(['min', 'max'])
   .assign(const=1).dot(beta_hat), color='orange')` :
   matplotlib 라이브러리의 `plot` 함수를 사용하여
   예측 값의 선을 그립니다.
   `df_berry2[['fruitset']].agg(['min', 'max'])`는
   'fruitset' 열의 최솟값과 최댓값을 가진 데이터프레임을 생성합니다.
   그런 다음 `.assign(const=1)`을 사용하여 상수 항을 추가합니다.
   `.dot(beta_hat)`은 예측 값을 계산합니다.
   이때 `beta_hat`은 회귀 계수를 나타내며,
   점곱(dot product)을 통해 예측 값을 계산합니다.
   이렇게 계산된 예측 값은 오렌지색 선으로 플롯됩니다.

따라서 이 코드는 주어진 데이터프레임의 'fruitset'과 'yield' 열에 대한
산점도를 그리고, 선형 회귀 모델의 예측 값을
오렌지색 선으로 나타내어 시각화하는 작업을 수행합니다.
'''

sns.scatterplot(df_berry2, x='fruitset', y='yield', s = 15, alpha=0.5)
plt.plot(
    df_berry2[['fruitset']].agg(['min', 'max']),
    df_berry2[['fruitset']].agg(['min', 'max']).assign(const=1).dot(beta_hat),
    color = 'orange'
```

```
)  
plt.show()
```



선형 회귀 분석 모듈:

statsmodels.api.OLS, statsmodels.formula.api.ols

```
In [9]: from statsmodels.api import OLS
```

```
lm = OLS(y, X).fit()  
lm.summary()
```

Out[9]:

OLS Regression Results

Dep. Variable:	yield	R-squared:	0.989
Model:	OLS	Adj. R-squared:	0.989
Method:	Least Squares	F-statistic:	6515.
Date:	Wed, 06 Mar 2024	Prob (F-statistic):	6.82e-74
Time:	01:07:59	Log-Likelihood:	-474.32
No. Observations:	76	AIC:	952.6
Df Residuals:	74	BIC:	957.3
Df Model:	1		
Covariance Type:	nonrobust		

	coef	std err	t	P> t	[0.025	0.975]
fruitset	1.866e+04	231.246	80.715	0.000	1.82e+04	1.91e+04
const	-3595.3886	127.259	-28.253	0.000	-3848.957	-3341.820

Omnibus:	1.925	Durbin-Watson:	2.471
Prob(Omnibus):	0.382	Jarque-Bera (JB):	1.934
Skew:	0.351	Prob(JB):	0.380
Kurtosis:	2.656	Cond. No.	20.8

Warnings:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

```
In [10]: from statsmodels.formula.api import ols
# patsy formula를 통해 종속변수와 독립 변수를 정의합니다.
lm = ols('yield_ ~ fruitset',
```

```
data=df_berry2.rename(columns={'yield': 'yield_'}).fit()  
lm.summary()
```

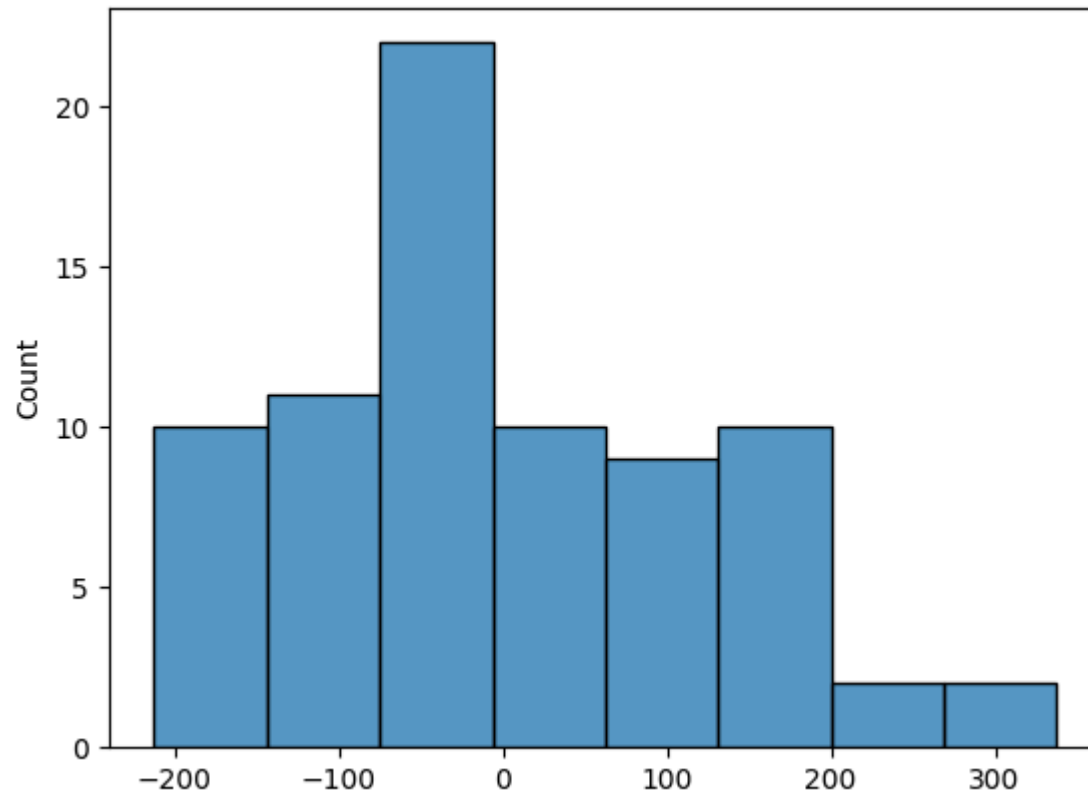
Out[10]:

OLS Regression Results						
Dep. Variable:	yield_		R-squared:	0.989		
Model:	OLS		Adj. R-squared:	0.989		
Method:	Least Squares		F-statistic:	6515.		
Date:	Wed, 06 Mar 2024		Prob (F-statistic):	6.82e-74		
Time:	01:07:59		Log-Likelihood:	-474.32		
No. Observations:	76		AIC:	952.6		
Df Residuals:	74		BIC:	957.3		
Df Model:	1					
Covariance Type:	nonrobust					
	coef	std err	t	P> t	[0.025	0.975]
Intercept	-3595.3886	127.259	-28.253	0.000	-3848.957	-3341.820
fruitset	1.866e+04	231.246	80.715	0.000	1.82e+04	1.91e+04
Omnibus:	1.925	Durbin-Watson:	2.471			
Prob(Omnibus):	0.382	Jarque-Bera (JB):	1.934			
Skew:	0.351	Prob(JB):	0.380			
Kurtosis:	2.656	Cond. No.	20.8			

Warnings:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

```
In [14]: fig, ax  
sns.histplot(lm.resid)  
plt.show()
```



```
In [5]: Image('101.png')
```

Out[5]: **2. 적합도(The goodness-of-fit)**

$$\sum_{i=1}^n (y_i - \bar{y})^2 = \sum_{i=1}^n (\hat{y}_i - \bar{y})^2 + \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

$$SST = SSR + SSE$$

SST (Sum of squares Total, TSS: Total sum of squares): 전체 변동성 - 평균에서 얼마나 데이터들이 떨어져 있는지는 나타냅니다.

SSR (Sum of Squared Regression, ESS: Explained sum of squares): 모델에 의해 설명된 변동성

SSE (Sum of squares of errors, RSS: Residual sum of squares): 모델에 의해 설명되지 못한 변동성

증명

$$SST = \|y - \bar{y}1\|^2 = \|y - \hat{y} + \hat{y} - \bar{y}1\|^2$$

$$= \|y - \hat{y}\|^2 + \|\hat{y} - \bar{y}1\|^2 + 2 \langle y - \hat{y}, \hat{y} - \bar{y}1 \rangle$$

$$= SSE + SSR + 2(\hat{e}^T X)\hat{\beta} - 2\bar{y}\hat{e}^T 1, \hat{e} = y - \hat{y}$$

$$(\hat{e}^T X) = (y - \hat{y})^T X = (y - X(X^T X)^{-1} X^T y)^T X = y^T X - y^T X(X^T X)^{-1} X^T X = 0$$

$$\hat{e}^T 1 = \sum_{i=1}^n y_i - \hat{y}_i = 0$$

$$= SSE + SSR$$

결정 계수(Coefficient of determination)

$$R^2 = \frac{SSR}{SST} = 1 - \frac{SSE}{SST}: \text{전체 변동성에서 회귀식에 의해 설명된 변동성의 비율}$$

[Ex.2]

위 모델의 SST, SSR, SSE를 구해 봅니다.

```
In [10]: s_prd = df_berry2[['fruitset']].assign(const=1).dot(beta_hat)
sse = ((df_berry2['yield'] - s_prd) ** 2).sum()
ssr = ((s_prd - df_berry2['yield'].mean()) ** 2).sum()
sst = ((df_berry2['yield'] - df_berry2['yield'].mean()) ** 2).sum()
```

```
In [11]: sst, ssr, sse, ssr + sse
```

```
Out[11]: (104426682.46588431,
103253862.22145344,
1172820.2444328119,
104426682.46588625)
```

```
In [12]: r2_1 = ssr / sst
r2_2 = 1 - sse / sst
r2_1, r2_2
```

```
Out[12]: (0.9887689600326619, 0.9887689600326434)
```

```
In [13]: print(lm.ssr, lm.ess)
```

```
1172820.2444328114 103253862.2214515
```

```
In [6]: Image('102.png')
```

```
Out[6]: 수정된 결정 계수
```

R^2 는 독립 변수가 종속 변수와 실제로 관련이 없더라도, 독립 변수의 수를 많이 가져갈 수록 높아지는 경향이 있습니다.

(p개의 모수(회귀 계수 + 절편)를 지닌 선형모델은 적어도 p개의 점은 오차 없이 예측할 수 있습니다.)

따라서 회귀 계수의 모수의 수에 따른 패널티를 가하여 결정계수를 수정한 것이 수정된 결정 계수 입니다.

$$AdjustedR^2 = 1 - \frac{MSE}{MST} = 1 - \frac{SSE/(n-p)}{SST/(n-1)}$$

```
In [14]: p = 2
n = len(df_berry2)
print("Adjusted R2", 1 - (sse / (n - p)) / (sst / (n - 1)))
```

Adjusted R2 0.9886171892222736

```
In [7]: Image('103.png')
```

Out[7]: **회귀 모델의 유의성 검정**

SSR(모델이 설명한 변동성), SSE(모델이 설명하지 못한 변동성)을 이용한 F-test

H_0 : 모든 회귀계수(절편 제외)는 0이다. ($\beta_1 = \beta_2 = \dots \beta_k = 0$) $\equiv MSR \leq MSE$, 회귀 모델은 유의미 하지 않다.

H_1 : 회귀계수(절편 제외)중 적어도 하나는 0이 아니다. ($\beta_1 \neq 0$ or $\beta_2 \neq 0 \dots$ or $\beta_k \neq 0$) $\equiv MSR > MSE$, 회귀 모델은 유의미하다.

$MSR = \frac{SSR}{df_R}$, 회귀모델의 모수의 수가 p일 때, 평균에 의해 고정되므로 1개가 차감됩니다. 따라서 p - 1

$MSE = \frac{SSE}{df_E}$, 모수가 p개는 p개의 표본에 대해서는 오차가 0으로 고정이 됩니다. 따라서 MSE p개 만큼의 표본이 고정되므로 자유도는 n - p 가 됩니다.

$$F = \frac{MSR}{MSE} \sim F(p - 1, n - p)$$

[Ex.3]

위 모델의 F-test를 수행해봅니다.

```
In [15]: from scipy.stats import f
n = len(df_berry2)
df_ssr = 1
df_sse = n - 2
F = (ssr / df_ssr) / (sse / df_sse)
pvalue = f.sf(F, df_ssr, df_sse)
print("SSR 자유도: {}, SSE 자유도: {}, 검정통계량: {}, pvalue: {}".format(df_ssr, df_sse, F, pvalue))
```

SSR 자유도: 1, SSE 자유도: 74, 검정통계량: 6514.882259798234, pvalue: 6.819989886213649e-74

```
In [16]: print(lm.fvalue, lm.f_pvalue)
```

```
6514.882259798114 6.8199898862182484e-74
```

```
In [8]: Image('104.png')
```

Out[8]: 회귀 계수의 유의성 검정

$$H_0 : \beta_i = 0$$

$$H_1 : \beta_i \neq 0$$

H_0 가 맞다면,

$$\mu_{\hat{\beta}_i}^{\wedge} = 0$$

$$s_{\hat{\beta}_i}^{\wedge} = \sqrt{\frac{\frac{1}{n-p} \sum_{i=1}^n \hat{e}_i^2}{\sum_{i=1}^n (x_i - \bar{x})^2}}$$

$$t = \frac{\hat{\beta}_i - \mu_{\hat{\beta}_i}^{\wedge}}{s_{\hat{\beta}_i}^{\wedge}} = \frac{\hat{\beta}_i}{s_{\hat{\beta}_i}^{\wedge}} \sim t(n-p)$$

[Ex.4]

위 모델의 회귀계수에 대한 회귀 계수의 유의성 검정을 해봅시다.

```
In [17]: n = len(df_berry2)
p = 2

s_beta_hat = ((sse / (n - p)) /
              ((df_berry2['fruitset'] - df_berry2['fruitset'].mean()) ** 2)
              .sum()) ** 0.5

s_beta_hat
```

Out[17]: 231.2455727733877

```
In [18]: from scipy.stats import t

# 양측 검정이므로 왼쪽의 꼬리영역을 모든 경우에 구하도록 -abs를 통계량에 적용합니다.
t_stat = -abs(beta_hat[0] - 0) / s_beta_hat
print("검정통계량: {}, pvalue: {}".format(t_stat, t.cdf(t_stat, n - p) * 2))
```

검정통계량: -80.71482057093502, pvalue: 6.819989886213796e-74

```
In [19]: lm.tvalues, lm.pvalues
```

```
Out[19]: (Intercept    -28.252552
fruitset         80.714821
dtype: float64,
Intercept         2.204123e-41
fruitset          6.819990e-74
dtype: float64)
```

```
In [9]: Image('105.png')
```

Out[9]:

3. 이상점과 영향점

이상점: 보편적인 분포에 벗어난 관측치

영향점: 이상점 중에서 회귀 분석 결과에 큰 영향을 미치는 소수의 관측치

이상점 판별법

1. 시각화
2. 점 하나씩 제외해보고, 회귀계수의 변화의 정도를 측정
3. 통계량

레버리지(Leverage): i번째 관측치와 평균 관측치의 거리, i번째 종속변수가 i번째 적합값(fitted value)에 미치는 영향도

$h_{ii} = x_i^T (X^T X)^{-1} x_i$, X 는 디자인 행렬, x_i 는 i번째 관측치에 해당하는 벡터

$$0 \leq h_{ii} \leq 1$$

표준화 잔차(studentized residual): i번째 관측치의 상대적 잔차

$$t_i = \frac{\hat{e}_i}{\sqrt{MSE(1-h_{ii})}}$$

DFFITs(difference in fit(s)): i번째 관측치를 제거했을 때 적합값(fitted value)의 변동량

$$DFFITs = \frac{\hat{Y}_i - \hat{Y}_{i(i)}}{\sqrt{MSE_{(i)} h_{ii}}}$$

$\hat{Y}_{i(i)}$: i번째 관측치를 제외하고 적합시킨 회귀모델의 적합값(fitted value)

$MSE_{(i)}$: i번째 관측치를 제외하고 회귀모델을 적합시켰을 때의 MSE

DFBETAS(difference in beta(s)): i번째 관측치를 제거했을 때, 회귀계수의 변동량

$$DFBETAS_{ij} = \frac{\hat{\beta}_j - \hat{\beta}_{(i)j}}{\sqrt{MSE_{(i)} C_{jj}}}$$

C_{jj} : $(X^T X)^{(-1)}$ 의 대각 요소중 j번째 값

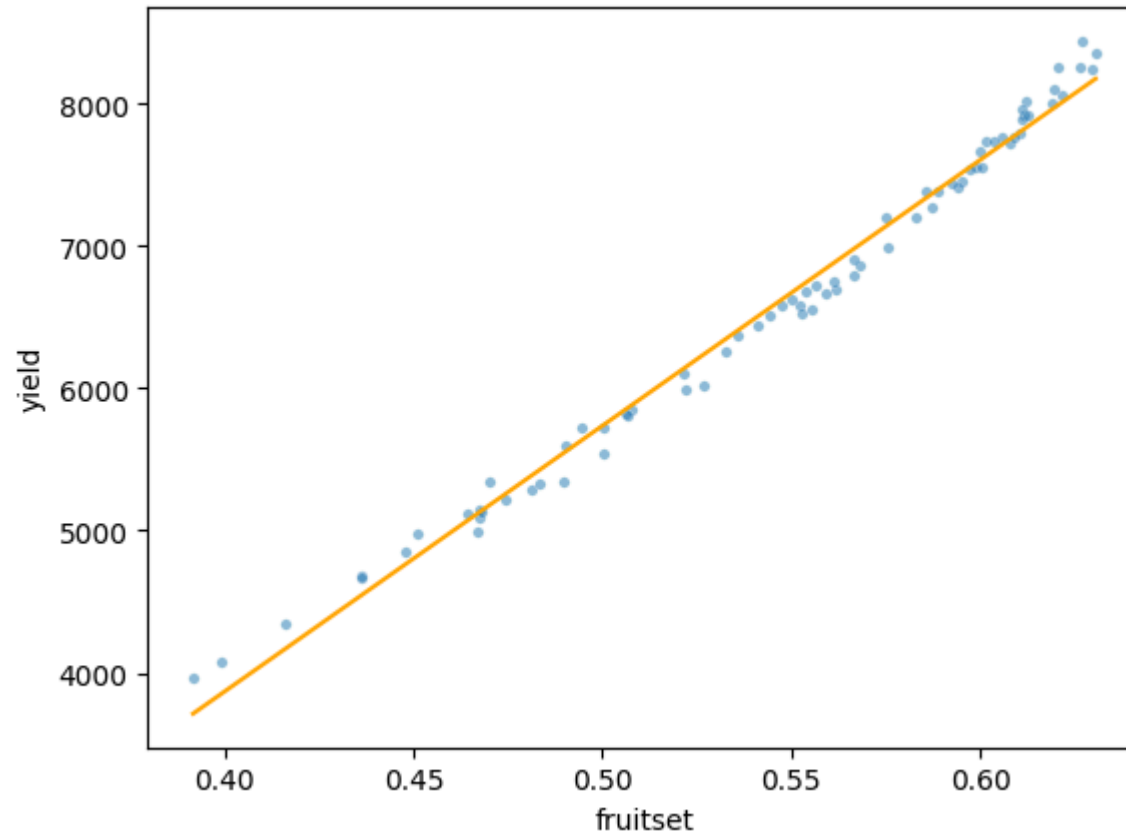
[Ex.5]

위 모델에서 영향점 분석을 해봅니다.

```
In [20]: sns.scatterplot(df_berry2, x='fruitset', y='yield', s = 15, alpha=0.5)

plt.plot(
    df_berry2[['fruitset']].agg(['min', 'max']),
    df_berry2[['fruitset']].agg(['min', 'max']).assign(const=1).dot(beta_hat),
    color = 'orange'
)

plt.show()
```



Leverage

h_i : i 번째 요소가 h_{ii} 인 벡터, H 를 구해 봅니다.

```
In [21]: '''
코드는 다음 작업을 수행합니다:

1. `np.dot(X.T, X)` : 행렬  $X$ 의 전치행렬과  $X$ 를 곱합니다.
   이는  $X$ 의 제곱합(Sum of Squares, SS) 행렬을 생성합니다.
2. `np.linalg.inv(...)` :
   앞서 계산한 제곱합 행렬의 역행렬을 계산합니다.
   이를 통해 최소 제곱 추정량을 계산하는데 사용됩니다.
3. `np.dot(np.dot(X, np.linalg.inv(np.dot(X.T, X))), X.T)` :
   앞서 계산한 제곱합 행렬의 역행렬과  $X$ 를 곱하고,
```

그 결과에 다시 X 의 전치행렬을 곱합니다.
 이는 행렬 X 에 대한 투영 행렬(projection matrix)을 생성합니다.
 투영 행렬은 다차원 공간에서 저차원 하위공간에
 데이터를 투영하는데 사용됩니다.

4. `np.diagonal(...)`:

투영 행렬의 대각선 요소를 추출합니다.
 이는 행렬의 대각선 요소만을 가져오는 작업으로,
 각 관측치가 자기 자신에 대한 투영된 값(예측값)을 나타냅니다.

따라서 코드는 행렬 X 의 자기 자신에 대한 투영된 값(예측값)을 추출하여
 배열 h 에 할당합니다. 이렇게 함으로써 선형 회귀 모델에서
 각 관측치의 예측값을 구하는 것을 가능케 합니다.

...

```
h = np.diagonal(
    np.dot(
        np.dot(X, np.linalg.inv(np.dot(X.T, X))), X.T
    )
)
h
```

```
Out[21]: array([0.01523545, 0.01393721, 0.03463865, 0.0943925 , 0.01325499,
                0.0186333 , 0.02409748, 0.07061227, 0.01596129, 0.02270438,
                0.01829397, 0.05457602, 0.02109729, 0.03449118, 0.01327086,
                0.03431945, 0.02739609, 0.03631289, 0.01367082, 0.03094264,
                0.0145278 , 0.01386002, 0.03434626, 0.08663494, 0.01332159,
                0.02080256, 0.02047877, 0.0545788 , 0.01755107, 0.02486817,
                0.01530754, 0.04629064, 0.02237219, 0.03212922, 0.01328602,
                0.03307878, 0.02567748, 0.03689312, 0.01346479, 0.02390718,
                0.01820326, 0.02678714, 0.01355782, 0.0440468 , 0.02169312,
                0.03098493, 0.01319579, 0.02776659, 0.02619731, 0.0307663 ,
                0.01467346, 0.01873598, 0.01589332, 0.02321239, 0.01383219,
                0.03612474, 0.02284276, 0.02751387, 0.0131588 , 0.02676944,
                0.02709694, 0.03475727, 0.01449145, 0.01856223, 0.01917294,
                0.02403578, 0.01318245, 0.03422745, 0.02030775, 0.02707699,
                0.01339857, 0.02234441, 0.02779673, 0.03144839, 0.01445216,
                0.0204768 ])
```

표준화 잔차를 구해봅니다.

t_{std} : i 번째 요소가 t_i 인 벡터를 만들어 봅니다.

```
In [22]: mse = sse / (n - 2)
t_std = (df_berry2['yield'] - s_prd) / np.sqrt((mse * (1 - h)))
t_std
```

```
Out[22]: 51    -1.206325
52    -1.620375
55    -0.982278
56     2.124771
67    -1.054218
...
328    0.752778
339    0.602520
340    2.178132
343   -1.406458
344   -0.088764
Length: 76, dtype: float64
```

```
In [23]: # statmodels의 영향점 분석결과입니다.
influence = lm.get_influence()

df_influence = pd.concat([
    pd.Series(h, name='leverages'),
    pd.Series(influence.resid_studentized, name='studentized residuals'),
    pd.Series(influence.dffits[0], name='dffits'),
    pd.DataFrame(influence.dfbetas, columns=['beta_hat_0', 'beta_hat_1']),
], axis=1).set_index(df_berry2.index)
df_influence
```

Out[23]:

	leverages	studentized residuals	dffits	beta_hat_0	beta_hat_1
51	0.015235	-1.206325	-0.150516	-0.071096	0.055582
52	0.013937	-1.620375	-0.194823	0.024291	-0.046069
55	0.034639	-0.982278	-0.186022	-0.158555	0.146491
56	0.094393	2.124771	0.703114	0.677846	-0.652270
67	0.013255	-1.054218	-0.122278	-0.003427	-0.010466
...	...	...	...	...	...
328	0.022344	0.752778	0.113468	0.082166	-0.072755
339	0.027797	0.602520	0.101439	-0.065219	0.073614
340	0.031448	2.178132	0.402954	-0.275743	0.307305
343	0.014452	-1.406458	-0.171468	0.032416	-0.051313
344	0.020477	-0.088764	-0.012748	-0.008731	0.007621

76 rows × 5 columns

```
In [24]: fig, axes = plt.subplots(1, 3, figsize=(16, 4))

for val, i, ax in zip(
    [influence.resid_studentized, influence.dffits[0], influence.dfbetas[:, 1]],
    ['Studentized residuals', 'DFFITs', 'DFBetas(beta_1)'],
    axes
):
    sns.scatterplot(
        x=df_berry2['fruitset'].values,
        y=df_berry2['yield'].values, s = 15, alpha=0.5,
        hue=val, ax=ax
    )

    ax.plot(
        df_berry2[['fruitset']].agg(['min', 'max']),
```

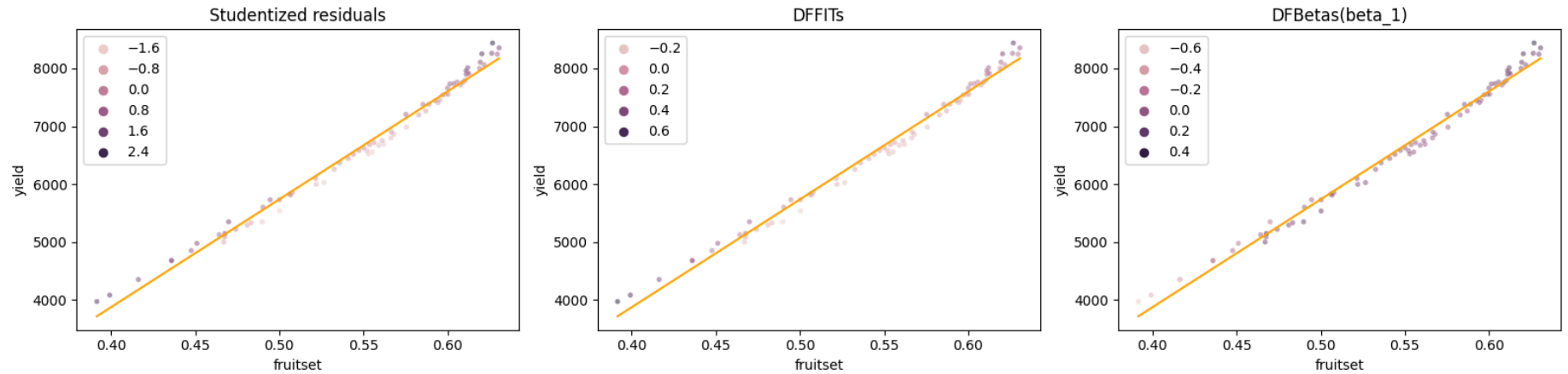
```

df_berry2[['fruitset']].agg(['min', 'max']).assign(const=1).dot(beta_hat),
color = 'orange'
)

ax.set_title(i)
ax.set_xlabel('fruitset')
ax.set_ylabel('yield')

plt.tight_layout()
plt.show()

```



In [10]: Image('106.png')

Out[10]: **다중 공선성(multicollinearity)**

- 다중공선성은 다중 선형 회귀 모델의 독립 변수간에 발생하는 문제입니다.
- 다중 선형 회귀에서 독립변수들 간에 강한 선형관계가 존재하는 경우를 말합니다.

다중 선형 회귀(Multiple Linear Regression)

- 독립 변수의 수가 여러 개인 선형 회귀 모델

※ 다중 선형 회귀 분석의 표준화 또는 정규화

독립 변수의 변수폭(스케일, Scale)을 균일화를 시켜 계수의 크기로 종속변수에 주는 영향도의 비교를 용이하게 합니다.

다중공선성의 영향

- 불안정한 모델: 회귀계수 추정량의 변동성이 커져, 데이터가 조금만 바뀌어도 변동성이 커집니다.
- 완전한 다중공선성: 독립변수들 간에 완벽한 선형 관계가 있는 경우

OLS: $\hat{\beta} = (X^T X)^{-1} X^T y$ 에서 $(X^T X)^{-1}$ 가 존재하지 않습니다.

제대로된 회귀 계수를 얻을 수 없어, 반드시 해소해야 합니다.

Ex) 범주형 변수를 가변수화 하면 이들 간에 완벽한 공선성을 지니게 됩니다. 가변수 중 하나를 제거합니다.

다중공선성 진단

- 시각화: Ex) 독립 변수간에 산점도 출력,
- 상관행렬 도출
- 독립 변수 마다 VIF(variance inflation factor) 측정

세경 리

애설 공간

- 변수 선택
- 차원 축소

[Ex.6]

df_berry2에서 yield 는 종속 변수 fruitset, fruitmass, seed은 독립 변수로 회귀 모델을 만들어 봅니다.

fruitset, fruitmass, seed를 표준화 합니다.

```
In [25]: from scipy.stats import zscore
        from statsmodels.api import add_constant
```

```
In [26]: X_cols = ['fruitmass', 'fruitset', 'seeds']
        X2 = add_constant(zscore(df_berry2[X_cols]))
```

```
In [27]: np.dot(np.linalg.inv(X2.T.dot(X2)), np.dot(X2.T, y))
```

```
Out[27]: array([ 6609.94253637, -2331.35215554,   940.43623399,  2564.02706691])
```

```
In [28]: lm2 = OLS(y, X2).fit()
        lm2.summary()
```

Out[28]:

OLS Regression Results						
Dep. Variable:		yield		R-squared:		0.994
Model:		OLS		Adj. R-squared:		0.994
Method:		Least Squares		F-statistic:		4168.
Date:		Mon, 04 Mar 2024		Prob (F-statistic):		1.31e-80
Time:		18:39:26		Log-Likelihood:		-448.72
No. Observations:		76		AIC:		905.4
Df Residuals:		72		BIC:		914.8
Df Model:		3				
Covariance Type:		nonrobust				
	coef	std err	t	P> t	[0.025	0.975]
const	6609.9425	10.453	632.340	0.000	6589.105	6630.780
x1	-2331.3522	1396.210	-1.670	0.099	-5114.645	451.941
x2	940.4362	317.907	2.958	0.004	306.699	1574.173
x3	2564.0271	1096.803	2.338	0.022	377.590	4750.465
Omnibus:		3.761	Durbin-Watson:		1.771	
Prob(Omnibus):		0.153	Jarque-Bera (JB):		2.985	
Skew:		0.370	Prob(JB):		0.225	
Kurtosis:		3.628	Cond. No.		298.	

Warnings:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

[Ex.7]

df_berry2에서 yield 는 종속 변수 fruitset, fruitmass, seed은 독립 변수로 회귀 모델에서의 다중 공선성 현상을 조사합니다.

그리고 VIF를 기준으로 변수를 선택하여 해소하는 방법과,

PCA를 사용하여 해소하는 방법을 보입니다.

In [29]:

```
'''
코드는 주어진 데이터프레임 `df_berry2`에서
두 개의 독립변수(`X_cols`) 조합에 대한 산점도를 그리는 작업을 수행합니다.

1. `import itertools`:
    itertools 모듈을 임포트합니다. 이 모듈은 반복 가능한 객체(iterable)에
    대한 유용한 함수들을 제공합니다.

2. `fig, axes = plt.subplots(1, 3, figsize=(12, 4))`:
    하위 그래프(subplot)를 포함하는 Figure 객체와
    그 하위 그래프의 배열을 생성합니다.
    여기서 1행 3열의 그래프를 생성하며, 그 크기는 12x4인치입니다.

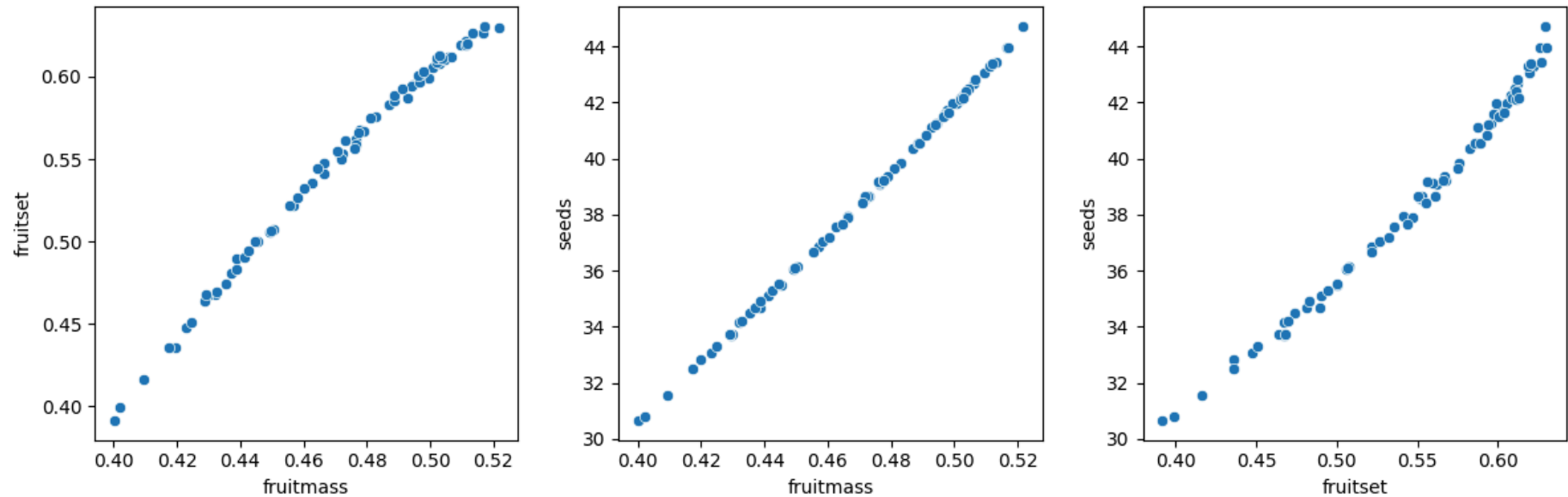
3. `for (x_, y_), ax in zip(itertools.combinations(X_cols, 2), axes):`:
    itertools 모듈의 `combinations` 함수를 사용하여
    `X_cols`의 요소들 중 두 개씩 조합한 조합들을 반환합니다.
    이 조합들에 대해 반복하면서 각 조합을 x와 y로 지정하고,
    이에 해당하는 축(ax)을 가져옵니다.

4. `sns.scatterplot(data=df_berry2, x=x_, y=y_, ax=ax)`:
    seaborn 라이브러리의 `scatterplot` 함수를 사용하여 산점도를 그립니다.
    `df_berry2`는 데이터프레임을 나타내며,
    x와 y에는 각각 현재 반복 중인 독립변수의 조합이 지정됩니다.
    `ax=ax`를 사용하여 각 하위 그래프에 해당하는 축을 지정합니다.

5. `plt.tight_layout()`:
    각 하위 그래프의 레이아웃을 조정하여 겹치지 않도록 합니다.

이렇게 하면 주어진 데이터프레임에서 가능한 모든 독립변수 조합에 대한
산점도가 생성되고, 이를 하나의 그림에 나란히 표시합니다.
'''
```

```
import itertools
fig, axes = plt.subplots(1, 3, figsize=(12, 4))
for (x_, y_), ax in zip(itertools.combinations(X_cols, 2), axes):
    sns.scatterplot(data=df_berry2, x=x_, y=y_, ax=ax)
plt.tight_layout()
plt.show()
```



In [30]:

```
'''
코드는 다중공선성(Multicollinearity)을 평가하기 위해
VIF(Variance Inflation Factor)를 계산합니다.

1. `vifs = [variance_inflation_factor(X2, i) for i in range(3)]:`
   `variance_inflation_factor` 함수를 사용하여 각 독립 변수의 VIF를 계산합니다.
   `X2`는 독립 변수 행렬을 나타내며, `i`는 각 독립 변수의 인덱스를 의미합니다.
   `range(3)`는 독립 변수의 수를 나타내는데,
   여기서는 0부터 2까지의 인덱스를 가진 독립 변수를 대상으로
   VIF를 계산합니다. 계산된 VIF 값들은 리스트 `vifs`에 저장됩니다.

2. `X_cols[np.argmax(vifs)]:`
   VIF 값이 가장 큰 독립 변수의 열 이름을 찾습니다.
   `np.argmax(vifs)`를 사용하여 VIF 값이 가장 큰 값의 인덱스를 찾은 후,
```

이를 사용하여 `X\_cols`에서 해당하는 독립 변수의 이름을 선택합니다.

따라서 코드는 주어진 독립 변수들에 대한 VIF 값을 계산하고,
그 중에서 VIF 값이 가장 큰 독립 변수의 이름을 반환합니다.
VIF 값이 높은 변수는 다른 독립 변수들과 높은 상관관계를 가지고 있어
다중공선성의 가능성이 있습니다.
'''

```
from statsmodels.stats.outliers_influence import variance_inflation_factor

vifs = [variance_inflation_factor(X2, i) for i in range(3)]
vifs, X_cols[np.argmax(vifs)]
```

Out[30]: ([1.0000000000000002, 17840.509291167236, 924.9268899020105], 'fruitset')

In [31]: '''

코드는 다음과 같은 작업을 수행합니다:

1. `zscore(df\_berry2[['fruitmass', 'seeds']])`:
'fruitmass'와 'seeds' 열에 대한 Z-score를 계산합니다.
Z-score는 각 변수의 값이 평균에서 표준편차의
몇 배만큼 떨어져 있는지를 나타내는 표준화된 값입니다.
2. `add\_constant(...)`:
독립 변수 행렬에 상수항(intercept)을 추가합니다.
이를 위해 statsmodels의 `add\_constant` 함수를 사용합니다.
상수항은 회귀 모델에서 절편을 나타냅니다.
3. `OLS(y, X3)`:
OLS(Ordinary Least Squares) 회귀 모델을 설정합니다.
종속 변수 `y`와 독립 변수 행렬 `X3`를 사용하여
선형 회귀 모델을 정의합니다.
4. `lm3.fit()`:
설정된 OLS 모델을 데이터에 적합시킵니다.
이를 통해 회귀 모델의 계수들이 추정됩니다.
5. `lm3.summary()`:
적합된 선형 회귀 모델의 요약 통계를 출력합니다.
이 요약에는 회귀 계수, t-통계량, p-value, 결정 계수 등의
정보가 포함됩니다.

따라서 코드는 주어진 독립 변수들을 사용하여
선형 회귀 모델을 적합하고,

해당 모델의 요약 통계를 출력합니다.
이를 통해 각 독립 변수가 종속 변수에 미치는 영향과
모델의 적합도를 평가할 수 있습니다.
'''

```
X3 = add_constant(zscore(df_berry2[['fruitmass', 'seeds'])))
```

```
lm3 = OLS(y, X3).fit()
```

```
lm3.summary()
```

Out[31]:

OLS Regression Results						
Dep. Variable:		yield		R-squared:		0.994
Model:		OLS		Adj. R-squared:		0.993
Method:		Least Squares		F-statistic:		5647.
Date:		Mon, 04 Mar 2024		Prob (F-statistic):		9.53e-81
Time:		18:39:27		Log-Likelihood:		-453.08
No. Observations:		76		AIC:		912.2
Df Residuals:		73		BIC:		919.1
Df Model:		2				
Covariance Type:		nonrobust				
	coef	std err	t	P> t	[0.025	0.975]
const	6609.9425	10.994	601.227	0.000	6588.031	6631.854
x1	1627.4627	418.716	3.887	0.000	792.964	2461.961
x2	-459.2605	418.716	-1.097	0.276	-1293.759	375.238
Omnibus:		0.366	Durbin-Watson:		1.559	
Prob(Omnibus):		0.833	Jarque-Bera (JB):		0.536	
Skew:		0.106	Prob(JB):		0.765	
Kurtosis:		2.648	Cond. No.		76.2	

Warnings:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

```
In [32]: vifs = [variance_inflation_factor(X3, i) for i in range(1, 3)]  
vifs
```

```
Out[32]: [1450.5047348236114, 1450.5047348236114]
```

```
In [33]: lm3 = OLS(y, df_berry2[['fruitset']].assign(const=1)).fit()  
lm3.rsquared_adj
```

```
Out[33]: 0.9886171892222737
```

```
In [34]: lm3 = OLS(y, df_berry2[['seeds']].assign(const=1)).fit()  
lm3.rsquared_adj
```

```
Out[34]: 0.9921447138547256
```

```
In [35]: lm3 = OLS(y, df_berry2[['seeds']].assign(const=1)).fit()  
lm3.summary()
```

Out[35]:

OLS Regression Results

Dep. Variable:		yield	R-squared:		0.992	
Model:		OLS	Adj. R-squared:		0.992	
Method:		Least Squares	F-statistic:		9474.	
Date:		Mon, 04 Mar 2024	Prob (F-statistic):		7.46e-80	
Time:		18:39:27	Log-Likelihood:		-460.22	
No. Observations:		76	AIC:		924.4	
Df Residuals:		74	BIC:		929.1	
Df Model:		1				
Covariance Type:		nonrobust				
	coef	std err	t	P> t	[0.025	0.975]
seeds	321.7481	3.306	97.333	0.000	315.162	328.335
const	-5783.7003	127.896	-45.222	0.000	-6038.539	-5528.861
Omnibus:		2.719	Durbin-Watson:		1.381	
Prob(Omnibus):		0.257	Jarque-Bera (JB):		2.273	
Skew:		-0.132	Prob(JB):		0.321	
Kurtosis:		3.805	Cond. No.		413.	

Warnings:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

```
In [36]: from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import make_pipeline
```

```
'''
```

코드는 주성분 분석(Principal Component Analysis, PCA)을 수행하여 주성분(PC)들이 데이터의 분산을 설명하는 비율을 계산합니다.

1. `pca = PCA()`:
PCA 객체를 생성합니다. 이 객체는 PCA 분석을 수행하는 데 사용됩니다.
2. `pca.fit(X2[:, :-1])`:
PCA 객체를 데이터에 적합시킵니다.
여기서 `X2[:, :-1]`은 독립 변수 행렬에서 상수항을 제외한 부분입니다.
PCA는 데이터의 공분산 행렬을 기반으로 주성분을 추출하기 때문에,
데이터를 적합할 때에는 상수항을 제거하는 것이 보편적입니다.
3. `pca.explained_variance_ratio_`:
`explained_variance_ratio_` 속성을 사용하여
각 주성분(PC)이 전체 분산에서 설명하는 비율을 계산합니다.
이는 주성분들이 데이터의 분산을 설명하는 정도를 나타내는데,
값이 클수록 해당 주성분이 많은 정보를 가지고 있음을 의미합니다.

코드는 주어진 데이터에서 주성분 분석을 수행하고,
각 주성분이 데이터의 분산을 설명하는 비율을 계산합니다.
이를 통해 주성분이 데이터를 얼마나 잘 설명하는지를 확인할 수 있습니다.

```
'''
```

```
pca = PCA()
pca.fit(X2[:, :-1])
pca.explained_variance_ratio_
```

```
Out[36]: array([0.99794425, 0.00205575, 0.          ])
```

```
In [37]: pca = make_pipeline(StandardScaler(), PCA(n_components=1))
X_pca = pca.fit_transform(X2[:, :-1])
```

```
In [38]: from statsmodels.api import add_constant
lm3 = OLS(y, add_constant(X_pca)).fit()
lm3.summary()
```

Out[38]:

OLS Regression Results						
Dep. Variable:		yield			R-squared: 0.993	
Model:		OLS			Adj. R-squared: 0.993	
Method:		Least Squares			F-statistic: 1.075e+04	
Date:		Mon, 04 Mar 2024			Prob (F-statistic): 7.28e-82	
Time:		18:39:27			Log-Likelihood: -455.47	
No. Observations:		76			AIC: 914.9	
Df Residuals:		74			BIC: 919.6	
Df Model:		1				
Covariance Type:		nonrobust				
	coef	std err	t	P> t	[0.025	0.975]
const	6609.9425	11.269	586.570	0.000	6587.489	6632.396
x1	-826.8765	7.976	-103.665	0.000	-842.770	-810.983
Omnibus:		0.814	Durbin-Watson:		2.098	
Prob(Omnibus):		0.666	Jarque-Bera (JB):		0.896	
Skew:		0.227	Prob(JB):		0.639	
Kurtosis:		2.724	Cond. No.		1.41	

Warnings:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

In []: